

C 语言程序设计大作业

Codeforces Crawler 2025

单 位: 计算机学院

班 级: 计算机科学与技术一班

学 号: 202405567006

姓 名: 贺谦昊

任课教师: 谢勇

湘潭大学

2025 年 5 月

文档信息

源代码	可执行文件	自评成绩	答辩 PPT
有	有	优	有

学术诚信承诺书

贺谦昊, 学号 202405567006, 系信息工程学院计算机系 24 级计算机科学与技术专业 1 班学生. 本人承诺本课程设计提交的所有源代码和文档不存在抄袭、剽窃行为. 如果被他人发现存在抄袭、剽窃、雷同等学术不端行为, 愿意承担所带来的一切后果.

贺谦昊
2025 年 5 月

目录

目录	2
1 题目概述	3
1.1 概述	3
1.2 所要完成的任务说明	3
1.3 完成情况	4
1.4 开发环境说明	5
2 程序分析与设计	5
2.1 系统分析	5
2.2 函数设计	13
3 系统演示	13
3.1 系统各功能运行截图与说明	13
4 设计体会	15
4.1 设计过程中的问题及其解决方案	15
4.2 设计过程中的错误及其原因分析	15
4.3 体会和收获	16

1 题目概述

1.1 概述

本项目旨在设计并实现一个功能完善的爬取 CF 的爬虫。该程序的核心功能是利用 Codeforces 官网提供的 API, 自动化地抓取指定用户的公开信息。通过对原始数据进行一系列的处理、分析和统计, 程序最终会将结果以结构化数据的形式输出, 并进一步利用这些数据生成动态、交互式的可视化图表, 以使用户能够在浏览器中直观地回顾和分析其在过去一年内的比赛历程和成长轨迹。

1.2 所要完成的任务说明

根据项目要求, 主要任务分为必做功能和选作功能两大类, 具体如下表所示:

序号	功能分类	任务描述
B1	必做	读取和解析指定用户的基本信息.
B2	必做	获取并筛选出过去一年内 Codeforces 官方举办的所有 Div.1, Div.2, Div.3, Div.4 级别的常规比赛列表.
B3	必做	读取指定用户在过去一年内实际参加的比赛列表, 并同样筛选出其中的 Div.1, Div.2, Div.3, Div.4 级别比赛.
B4	必做	对于用户参加的每一场符合条件的比赛, 详细读取其排名, 每道题目的得分情况, 比赛前后的 Rating 变化以及头衔变化.
B5	必做	统计并准备展示用户在过去一年内参加各级别 (Div.1-4) 比赛的总次数.
B6	必做	统计并准备展示用户在过去一年内参赛次数的月度分布情况.
B7	必做	计算并准备展示用户在过去一年内对于 Div.1-4 级别比赛的整体出勤率.
B8	必做	汇总并准备展示用户所有已获取的参赛记录, 包括比赛的级别、完整名称、比赛时间、个人排名、每道题目的得分详情、Rating 变化以及头衔变化.
B9	必做	统计并展示题目分值的分布.
S1	选作	分月统计所有 Div.1-4 级别比赛中, 各题目的通过准确性 (AC 率) 以及题目原始分值的分布, 并用图表展示.
S2	选作	深入分析用户在比赛中通过的题目, 统计其通过题目的难度分布, 以及尝试分析题目涉及的知识点分布, 并用图表展示.

1.3 完成情况

截至目前, 项目各项功能的完成情况如下表所示. 对于已完成的功能, 均已实现核心逻辑并通过了基本测试.

功能序号	完成情况说明
B1	已完成. 能够通过 API 成功获取并解析用户的 Handle、当前 Rating、最高 Rating, 并通过自定义函数映射为对应头衔. 数据已输出至 CSV.
B2	已完成. 能够获取 ‘contest.list’, 并根据比赛时间 (过去一年) 和名称中的”Div. X” 关键词筛选出符合条件的官方比赛列表. 数据已输出至 CSV.
B3	已完成. 能够通过 ‘user.rating’ API 获取用户所有 Rated 比赛记录, 并筛选出过去一年内用户参加的 Div.1-4 比赛.
B4	已完成. 通过结合 ‘user.rating’和对每场比赛调用 ‘contest standings’ API, 成功获取了用户在筛选后比赛中的排名、Rating 变化、头衔变化, 以及每道题目的得分和错误尝试次数. 数据已整合并输出至 CSV.
B5	已完成. 基于功能 B4 获取的数据, 在 C 程序中统计了用户参加各级别比赛的次数, 并输出到 CSV. Python 脚本可读取此 CSV 生成图表.
B6	已完成. 基于功能 B4 获取的数据, 在 C 程序中按月份统计了用户的参赛次数, 并输出到 CSV. Python 脚本可读取此 CSV 生成图表.
B7	已完成. 通过比较用户一年内参加的 Div.1-4 比赛数 (来自 B4 的数据) 和一年内总共举办的 Div.1-4 比赛数 (来自 B2 的数据), 计算得出出勤率, 并输出到 CSV.
B8	已完成. process_performances 函数已实现将 B1-B4 中获取和处理的比赛详细信息 (级别通过名称判断、名称、时间、排名、题目得分、Rating 变化、头衔变化) 汇总输出到核心的 _contest_performances.csv 文件中. 前端页面已能展示这些信息.
B9	部分完成/已调整. C 程序在输出 _contest_performances.csv 时, 包含了用户每题的实际得分和该场比赛的总题目数. 题目的原始满分值可以通过解析 contest standings 中的 problems 数组获得, 这部分已在 parse_contest_standings 中考虑, 但未在 CSV 中显式输出每题满分, 主要侧重用户实际得分. Python 脚本可基于用户得分进行统计. 对于 “全年比赛题目分值分布” 的全局统计尚未作为必做实现.
S1	未实现/初步探索. 全年所有比赛各题通过准确性和原始分值分布的统计需要大量的 API 调用和复杂的数据聚合, 目前尚未完整实现. 已具备获取全年比赛列表的基础.
S2	未实现. 用户通过题目的难度分和知识点分布分析需要更进一步的 API 数据 (如题目详情、标签) 和分析算法, 目前未进行.

1.4 开发环境说明

本项目主要在以下环境中进行开发与测试:

- **操作系统:** Linux Ubuntu 24.04 LTS
- **C 编译器:** GCC version 13.2.0
- **主要依赖库 (C):**
 - libcurl
 - cJSON
- **Python 环境:** Python 3.12.3
- **Python 使用库:**
 - Pandas
- **IDE:** Neovim
- **辅助工具:** GNU Make, Bash

C 程序通过 Makefile 进行编译. Python 脚本依赖 Pandas 库进行数据处理. 最终报告通过 Python 内置的 http.server 模块在本地启动 Web 服务进行查看.

2 程序分析与设计

2.1 系统分析

以下任意要求在文档中缺少, 不得分.

要求 1: 创建了哪些函数, 对函数分别进行详细说明.

说明要包含:

- 函数原型;
- 函数的功能;
- 各参数的意义;
- 函数的返回值类型.

表 1: 函数详细说明

编号	描述详情
001	函数原型: <code>int fetch_data(const char* url, MemoryStruct* chunk_ptr)</code> 函数功能: 使用 libcurl 向指定的 URL 发送 HTTP GET 请求, 并将获取到的数据存储在 <code>chunk_ptr</code> 指向的 <code>MemoryStruct</code> 结构中. 参数说明: <code>url</code>: <code>const char*</code> 型, 指向包含目标 URL 的字符串. <code>chunk_ptr</code>: <code>MemoryStruct*</code> 型, 指向一个 <code>MemoryStruct</code> 结构的指针, 用于存储下载的数据及其大小. 函数内部会为 <code>chunk_ptr->memory</code> 分配内存. 函数返回值: <code>int</code> 型, 例如返回 0 表示成功获取数据, 非 0 表示失败.
002	函数原型: <code>int parse_user_info(const MemoryStruct* response, UserInfo* user_info_ptr)</code> 函数功能: 解析从 Codeforces API 获取到的用户基本信息 JSON 响应, 并将解析后的数据填充到 <code>user_info_ptr</code> 指向的 <code>UserInfo</code> 结构中. 参数说明: <code>response</code>: <code>const MemoryStruct*</code> 型, 指向包含 JSON 响应数据的 <code>MemoryStruct</code> 结构. <code>user_info_ptr</code>: <code>UserInfo*</code> 型, 指向一个 <code>UserInfo</code> 结构的指针, 用于存储解析后的用户信息. 函数返回值: <code>int</code> 型, 例如返回 0 表示解析成功, 非 0 表示失败.
003	函数原型: <code>int parse_user_rating(const MemoryStruct* response, UserRatingChange ratings_list[], int max_ratings, int* count_ptr)</code> 函数功能: 解析从 Codeforces API 获取到的用户 Rating 变化历史 JSON 响应, 并将解析出的每条 Rating 变化记录存储到 <code>ratings_list</code> 数组中.

续下一页

表 1: 函数详细说明 (续)

编号	描述详情
	参数说明: response: const MemoryStruct* 型, 指向包含 JSON 响应数据的 MemoryStruct 结构. ratings_list[]: UserRatingChange 型数组, 用于存储解析后的 Rating 变化记录. max_ratings: int 型, ratings_list 数组的最大容量. count_ptr: int* 型, 指向一个整数的指针, 函数会将实际解析出的 Rating 变化记录数量存入此地址. 函数返回值: int 型, 例如返回 0 表示解析成功, 非 0 表示失败.
004	函数原型: int parse_contest_list(const MemoryStruct* response, ContestInfo contests[], int max_contests, int* count_ptr) 函数功能: 解析从 Codeforces API 获取到的比赛列表 JSON 响应, 并将解析出的每场比赛信息存储到 contests 数组中. 参数说明: response: const MemoryStruct* 型, 指向包含 JSON 响应数据的 MemoryStruct 结构. contests[]: ContestInfo 型数组, 用于存储解析后的比赛信息. max_contests: int 型, contests 数组的最大容量. count_ptr: int* 型, 指向一个整数的指针, 函数会将实际解析出的比赛数量存入此地址. 函数返回值: int 型, 例如返回 0 表示解析成功, 非 0 表示失败.
005	函数原型: int parse_contest_standings(const MemoryStruct* response, const char* handle, UserProblemPerformance* performance_entry) 函数功能: 解析从 Codeforces API 获取到的特定比赛的排名 (Standings) JSON 响应, 提取指定用户 (handle) 在该场比赛中的表现数据 (如 Rating 变化信息, 题目得分情况等), 并填充到 performance_entry 指向的 UserProblemPerformance 结构中.

续下一页

表 1: 函数详细说明 (续)

编号	描述详情
	参数说明: response: const MemoryStruct* 型, 指向包含比赛排名 JSON 响应数据的 MemoryStruct 结构. handle: const char* 型, 目标用户的 Codeforces Handle. performance_entry: UserProblemPerformance* 型, 指向一个 UserProblemPerformance 结构的指针, 用于存储解析后的用户比赛表现数据. 函数返回值: int 型, 例如返回 0 表示解析成功并找到用户数据, 非 0 表示失败或未找到用户.
006	函数原型: int filter_div1_4_ratings(const UserRatingChange source_ratings[], int source_size, UserRatingChange dest_ratings[], int limit) 函数功能: 从原始的用户 Rating 变化记录数组 source_ratings 中, 筛选出属于 Div.1, Div.2, Div.3, Div.4 级别的比赛记录, 并将符合条件的记录存入 dest_ratings 数组中. 通常通过检查 contestName 字段是否包含"Div. 1", "Div. 2", "Div. 3", "Div. 4" 等关键词来进行筛选. 参数说明: source_ratings[]: const UserRatingChange 型数组, 包含原始的 Rating 变化记录. source_size: int 型, source_ratings 数组中的记录数量. dest_ratings[]: UserRatingChange 型数组, 用于存储筛选后的记录. limit: int 型, dest_ratings 数组的最大容量. 函数返回值: int 型, 返回实际筛选出的符合条件的记录数量.
007	函数原型: int filter_div1_4_contests(const ContestInfo source_contests[], int source_size, ContestInfo dest_contests[], int limit)

续下一页

表 1: 函数详细说明 (续)

编号	描述详情
	函数功能: 从原始的比赛信息数组 <code>source_contests</code> 中, 筛选出属于 Div.1, Div.2, Div.3, Div.4 级别的比赛, 并将符合条件的比赛信息存入 <code>dest_contests</code> 数组中. 同样通过检查 <code>name</code> 字段是否包含 "Div. 1", "Div. 2", "Div. 3", "Div. 4" 等关键词来进行筛选. 参数说明: <code>source_contests[]</code>: <code>const ContestInfo</code> 型数组, 包含原始的比赛信息. <code>source_size</code>: <code>int</code> 型, <code>source_contests</code> 数组中的比赛数量. <code>dest_contests[]</code>: <code>ContestInfo</code> 型数组, 用于存储筛选后的比赛信息. <code>limit</code>: <code>int</code> 型, <code>dest_contests</code> 数组的最大容量. 函数返回值: <code>int</code> 型, 返回实际筛选出的符合条件的比赛数量.
008	函数原型: <code>void process_user_info(const UserInfo* user_info, const char* prefix)</code> 函数功能: 将解析后的用户信息 (存储在 <code>user_info</code> 结构中) 处理并输出到以 <code>prefix</code> 为前缀的特定文件中 (例如 CSV 文件). 参数说明: <code>user_info</code>: <code>const UserInfo*</code> 型, 指向包含用户信息的结构. <code>prefix</code>: <code>const char*</code> 型, 用于生成输出文件名的前缀. 函数返回值: <code>void</code> (无返回值).
009	函数原型: <code>void process_performances(const UserProblemPerformance performances[], int cnt, const char* prefix)</code> 函数功能: 将用户在一系列比赛中的表现数据 (存储在 <code>performances</code> 数组中) 处理并输出到以 <code>prefix</code> 为前缀的特定文件中 (例如核心的 <code>_contest_performances.csv</code> 文件). 参数说明: <code>performances[]</code>: <code>const UserProblemPerformance</code> 型数组, 包含用户在各场比赛中的表现数据.

续下一页

表 1: 函数详细说明 (续)

编号	描述详情
	cnt: int 型, performances 数组中的记录数量. prefix: const char* 型, 用于生成输出文件名的前缀. 函数返回值: void (无返回值).
010	函数原型: void process_division_stats(const UserProblemPerformance perf[], int cnt, const char* prefix) 函数功能: 根据用户的比赛表现数据, 统计其参加各级别 (Div.1-4) 比赛的次数, 并将统计结果输出到以 prefix 为前缀的文件中. 参数说明: perf[]: const UserProblemPerformance 型数组, 包含用户在各场比赛中的表现数据. cnt: int 型, perf 数组中的记录数量. prefix: const char* 型, 用于生成输出文件名的前缀. 函数返回值: void (无返回值).
011	函数原型: void process_monthly_stats(const UserProblemPerformance perf[], int cnt, const char* prefix) 函数功能: 根据用户的比赛表现数据, 按月份统计其参赛次数, 并将统计结果输出到以 prefix 为前缀的文件中. 参数说明: perf[]: const UserProblemPerformance 型数组, 包含用户在各场比赛中的表现数据. cnt: int 型, perf 数组中的记录数量. prefix: const char* 型, 用于生成输出文件名的前缀. 函数返回值: void (无返回值).
012	函数原型: void process_attendance_rate(int user_statistic, int total, const char* prefix)

续下一页

表 1: 函数详细说明 (续)

编号	描述详情
	函数功能: 根据用户年度参赛次数和年度总比赛次数, 计算并输出用户的出勤率到以 prefix 为前缀的文件中. 参数说明: user_statis: int 型, 用户在过去一年内参加的符合条件的比赛总次数. total: int 型, 过去一年内总共举办的符合条件的比赛总次数. prefix: const char* 型, 用于生成输出文件名的前缀. 函数返回值: void (无返回值).
013	函数原型: void wait_for_next_request() 函数功能: 实现一个等待机制, 用于在连续发送 API 请求之间引入延迟, 以避免因请求过于频繁而被 Codeforces API 服务器限制. 通常使用 sleep() 或类似的函数来实现固定时间的等待. 参数说明: 无. 函数返回值: void (无返回值).

要求 2: 函数之间的调用关系图

Mermaid 源代码可以在这里查看 <https://netcut.cn/1ise4h1kb>

图片链接 <https://i.111666.best/image/ffK2BBn7wshsyME1AR7y9d.png>

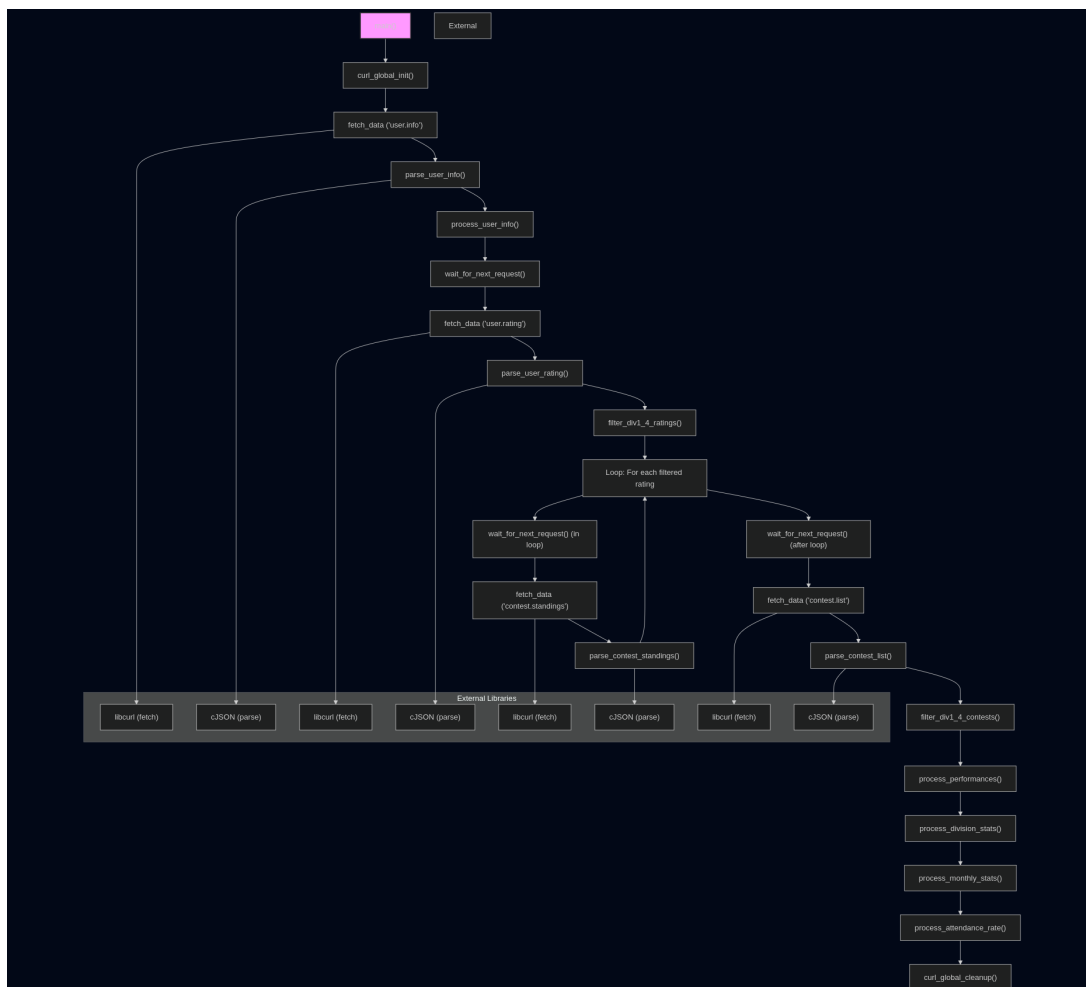


图 1: 函数调用关系图 (由 Mermaid 生成)

2.2 函数设计

相关文件由于代码太长，保存在另外一个文件。

3 系统演示

3.1 系统各功能运行截图与说明

```
~/c-spider 3.12.3 14:39:25
➔ ./run.sh fvone
[INFO] Processing for user: fvone
[INFO] Activating Python virtual environment...
[INFO] Compiling C program...
make: 'main' is up to date.
[INFO] Compilation finished.
[INFO] Running C program to generate CSV files for user 'fvone'...
[II] --Target user: fvone--
[II] --Initializing libcurl....--
[II] --libcurl initialized--

[II] --Fetching user info: https://codeforces.com/api/user.info?handles=fvone--
fetch_data: ready to fetch url: https://codeforces.com/api/user.info?handles=fvone
fetch_data: HTTP CODE: 200
fetch_data: success!
[II] --User info fetched (417 bytes)--
parse_user_info: Successfully parsed user info.
parse_user_info: Cleaning up JSON object...
[II] --User info parsed--

wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
[II] --Fetching user rating: https://codeforces.com/api/user.rating?handle=fvone--
fetch_data: ready to fetch url: https://codeforces.com/api/user.rating?handle=fvone
fetch_data: HTTP CODE: 200
fetch_data: success!
[II] --User rating fetched (1180 bytes)--
parse_user_rating: Found 7 rating change records in JSON.
[II] --User rating parsed (7 records)--
[II] --Filtered 7 Div. 1-4 ratings--

[II] ---Fetching Detailed Problem Scores---
```

图 2

```
[II] --Fetching standings: ID 2072 (Codeforces Round 1006 (Div. 3))...--
wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
fetch_data: ready to fetch url: https://codeforces.com/api/contest.standings?contestId=2072&handles=fvone&showUnofficial=true
fetch_data: HTTP CODE: 200
fetch_data: success!
[II] --Parsed standings for contest 2072--

[II] --Fetching standings: ID 2071 (Codeforces Round 1007 (Div. 2))...--
wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
fetch_data: ready to fetch url: https://codeforces.com/api/contest.standings?contestId=2071&handles=fvone&showUnofficial=true
fetch_data: HTTP CODE: 200
fetch_data: success!
[II] --Parsed standings for contest 2071--

[II] --Fetching standings: ID 2085 (Codeforces Round 1011 (Div. 2))...--
wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
fetch_data: ready to fetch url: https://codeforces.com/api/contest.standings?contestId=2085&handles=fvone&showUnofficial=true
fetch_data: HTTP CODE: 502
fetch_data: HTTP request failure, HTTP CODE: 502
[ER] --Failed to fetch standings for contest 2085--

[II] --Fetching standings: ID 2091 (Codeforces Round 1013 (Div. 3))...--
wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
fetch_data: ready to fetch url: https://codeforces.com/api/contest.standings?contestId=2091&handles=fvone&showUnofficial=true
fetch_data: HTTP CODE: 502
fetch_data: HTTP request failure, HTTP CODE: 502
[ER] --Failed to fetch standings for contest 2091--

[II] --Fetching standings: ID 2093 (Codeforces Round 1016 (Div. 3))...--
wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
fetch_data: ready to fetch url: https://codeforces.com/api/contest.standings?contestId=2093&handles=fvone&showUnofficial=true
fetch_data: HTTP CODE: 200
fetch_data: success!
```

图 3

```

wait_for_next_seconds: follow the API rate limit, need to wait 2 seconds...
[II] --Fetching contest list: https://codeforces.com/api/contest.list?gym=false--
fetch_data: ready to fetch url: https://codeforces.com/api/contest.list?gym=false
fetch_data: HTTP CODE: 200
fetch_data: success!
[II] --Contest list fetched (381952 bytes)--
parse_contest_list: Found 2006 contest records in JSON.
[II] --Contest list parsed (128 yearly contests before Div filter)--
[II] --Filtered 106 Div. 1-4 contests (last year)--

[II] ---Generating Statistics and Output Files---

----- User Info (Console) -----
User Handle: fvone
Current Rating: 1152 (Newbie)
Max Rating: 1167 (Newbie)
-----
[II] --User summary data written to csv/fvone_user_summary.csv--
[II] --Contest performances data written to csv/fvone_contest_performances.csv--

[II] --Participation Counts by Division (Based on 5 detailed performances):--
Div. 1: 0
Div. 2: 3
Div. 3: 2
Div. 4: 0
[II] --Division participation counts written to csv/fvone_division_counts.csv--

```

图 4

```

[II] --Monthly Participation (Based on 5 detailed performances):--
Jan: 0
Feb: 1
Mar: 1
Apr: 2
May: 1
Jun: 0
Jul: 0
Aug: 0
Sep: 0
Oct: 0
Nov: 0
Dec: 0
[II] --Monthly participation data written to csv/fvone_monthly_cal.csv--

[II] --Attendance Rate (Last Year):--
User attended 5 Div. 1-4 contests.
Total Div. 1-4 contests held: 106
Attendance Rate: 4.72%
[II] --Attendance rate data written to csv/fvone_attendance_rate.csv--

[II] ---Statistics Generation and Output Complete---

```

图 5

```

[II] --Cleaning up libcurl.--
Program complete.
Hope you have a nice day ~
[INFO] C program finished.
[INFO] Running Python script to generate ECharts JSON for user 'fvone'...
[II] -- Starting Python script for user 'fvone' at 2025-05-30 14:45:40 --
[II] -- Successfully loaded csv/fvone_user_summary.csv --
[II] -- Successfully loaded csv/fvone_contest_performances.csv --
[II] -- Successfully loaded csv/fvone_monthly_cal.csv --
[II] -- Successfully loaded csv/fvone_division_counts.csv --
[II] -- Successfully loaded csv/fvone_attendance_rate.csv --
[II] -- ECharts option saved to echarts_data/fvone_user_report_summary_option.json --
[II] -- ECharts option saved to echarts_data/fvone_rating_trend_option.json --
[II] -- ECharts option saved to echarts_data/fvone_monthly_cal_option.json --
[II] -- ECharts option saved to echarts_data/fvone_division_participation_option.json --
[II] -- ECharts option saved to echarts_data/fvone_problem_scores_recent_option.json --
[II] -- Python script for user 'fvone' finished at 2025-05-30 14:45:40 (Duration: 0:00:00.025378) --
[INFO] Python script finished.
[INFO] Starting HTTP server on port 8000...
[INFO] HTTP server started with PID: 8278. Access at http://localhost:8000/
[INFO] HTML report will be opened at: http://localhost:8000/index.html?user=fvone
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
[INFO] Opening report in default browser: http://localhost:8000/index.html?user=fvone

[IMPORTANT] The HTTP server is running in the background (PID: 8278).
[IMPORTANT] To stop the server, run: kill 8278
[IMPORTANT] Or press Ctrl+C in this terminal if you remove the '&' from the server command (but then the script won't open the browser).
[IMPORTANT] Alternatively, after closing the browser tab, you can press Ctrl+C in THIS terminal ONCE to stop the script and the server (due to the trap).
127.0.0.1 - - [30/May/2025 14:45:43] "GET /echarts_data/fvone_user_report_summary_option.json HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2025 14:45:43] "GET /echarts_data/fvone_rating_trend_option.json HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2025 14:45:43] "GET /echarts_data/fvone_monthly_cal_option.json HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2025 14:45:43] "GET /echarts_data/fvone_division_participation_option.json HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2025 14:45:43] "GET /echarts_data/fvone_problem_scores_recent_option.json HTTP/1.1" 200 -

```

图 6

4 设计体会

本次大作业的完成，让我感受到了上前行代码修改和维护起来的困难，也让我对工业级代码有了初步的体会。当然，也对一些工业上常见的 c 语言库有了一定的了解。总的来说，我认为这个大作业可以让新生第一次对大型项目的开发和维护有所了解，对新生来说纵使上手难度高，对自学和知识运用有较大考验，但确实对学生大有益处。

4.1 设计过程中的问题及其解决方案

在整个项目的设计与开发过程中，我遇到了一些预料之中及之外的技术难题和挑战。以下是一些主要问题及其对应的思考与解决过程：

- **问题 1：网络请求与数据获取的稳定性处理**

- 问题描述：项目过程中曾经由于大量对 cf 进行 API 访问，可能让 cloudflare 以为我这个 IP 有异，被短暂的 ban 一段时间。不过很快的恢复了。
- 解决方案：对于 API 速率限制，通过在连续请求之间加入固定的 `sleep()` 延时（即 `wait_for_next_request()` 函数）来主动控制请求频率，有效降低了被服务器拒绝的概率。

- **问题 2：JSON 数据解析的复杂性与健壮性**

- 问题描述：Codeforces API 返回的 JSON 数据结构有时较为复杂，且可能存在可选字段或嵌套层级较深的情况。初期使用 `cJSON` 解析时，如果某个预期字段不存在或数据类型不匹配，容易导致程序崩溃。
- 解决方案：在使用 `cJSON` 解析每个字段前，都添加了对该字段是否存在以及类型是否匹配（`cJSON_IsString`, `cJSON_IsNumber` 等）的检查。对于可选字段，如果不存在则赋予默认值或标记为空。对于数组解析，严格检查了数组大小和元素类型，增强了 JSON 解析的容错性和程序的健壮性。

4.2 设计过程中的错误及其原因分析

在开发过程中，调试和错误排查占据了相当一部分时间。一些典型的错误及其反思如下：

- **错误 1：内存泄漏与野指针问题**

- 表现：程序在长时间运行或处理大量数据后出现不稳定，甚至崩溃。
- 原因分析：主要是在使用 `libcurl` 的 `MemoryStruct` 以及 `cJSON` 动态分配内存后，部分分支路径或错误处理流程中忘记了调用 `free()` 释放内存。此外，也存在对已释放内存的指针进行访问的情况。

- 修正与反思：对每一个 `malloc` 或 `cJSON_Parse` 等分配内存的操作，都仔细检查了其对应的释放逻辑，确保在各种执行路径下都能正确释放。深刻理解了 C 语言内存管理的“谁申请，谁释放”原则。

• 错误 2：字符串处理不当导致的缓冲区溢出

- 表现：程序在拼接 URL、文件名、比赛名称时，出现段错误或输出异常。
- 原因分析：初期使用了 `strcpy`、`sprintf` 等函数，但未严格检查目标缓冲区的大小，当源字符串长度超过预期时，导致了缓冲区溢出。
- 修正与反思：改用了更安全的 `strncpy`、`snprintf`，并始终明确指定目标缓冲区的最大长度。对于动态长度的字符串，优先考虑动态内存分配，或者在拼接前计算所需总长度，确保分配足够的空间。加强了对字符串边界条件的关注。

• 错误 3：API 数据逻辑理解偏差

- 表现：计算用户 Rating 变化或统计比赛次数时，结果与 Codeforces 网站显示不符。
- 原因分析：初期对 `user.rating` API 返回结果中比赛类型的筛选逻辑不完善。
- 修正与反思：重新仔细阅读了 cf API 文档，并且利用浏览器直观浏览 json 文件格式，修正了数据提取和计算逻辑。认识到做项目先看 doc. 的重要性。

4.3 体会和收获

通过本次课程设计，我获益匪浅，主要体现在以下几个方面：

1. **C 语言编程能力的深化：**本项目让我有机会将学到的 C 语言知识 (`point`、`struct`、`file stream`、`memory management` etc.) 应用于一个相对复杂的实际问题中，极大地提升了我的代码实践能力和调试技巧。尤其是在内存管理和字符串处理方面，通过解决实际遇到的错误，理解更为深刻。但是 c 语言没有类似 java 或者 py 的完善的内存管理机制令我非常伤脑筋，要是能够有 RAII 也好 ()。
2. **第三方库的使用经验：**首次系统地学习并使用了 `libcurl` 和 `cJSON` 这两个重要的第三方 C 库。从阅读文档、编译链接到实际调用 API 进行网络请求和 JSON 解析，这个过程让我掌握了在 C 项目中集成和使用外部库的基本流程，拓宽了 C 语言的应用场景。

3. **问题解决与自主学习能力的提升：**在遇到技术难题或 API 使用不明确时，践行了 'STFW' , 'STFM' 原则，这些技术论坛和网站很好的帮助了我 (Stack Overflow, cpp reference)。
4. **对 Web API 和数据处理流程的理解：**通过实际调用 Codeforces API 并处理返回的 JSON 数据，我对 Web API 的工作原理、HTTP 请求、JSON 数据格式以及基本的数据清洗、转换和存储流程有了更直观的认识。
5. **文档撰写与总结反思的重要性：**编写课程设计报告的过程，也是对整个项目进行梳理、总结和反思的过程。通过文字将设计思路、遇到的问题和解决方案清晰地表达出来，不仅加深了对项目的理解，也锻炼了技术文档的写作能力。意识到清晰的注释和文档对于项目本身和后续维护同样重要。

总而言之，这次大作业不仅是对我 C 语言技能的一次全面检验和提升，更是一次宝贵的独立项目经验。虽然过程中遇到了不少困难，但克服它们所带来的成就感和学到的知识，将对我未来的学习和发展产生积极而深远的影响。我也认识到自己在某些方面（如更复杂的算法设计、大规模数据处理的性能优化、完备的错误处理和日志系统等）还有提升空间，这些都将是未来努力的方向。